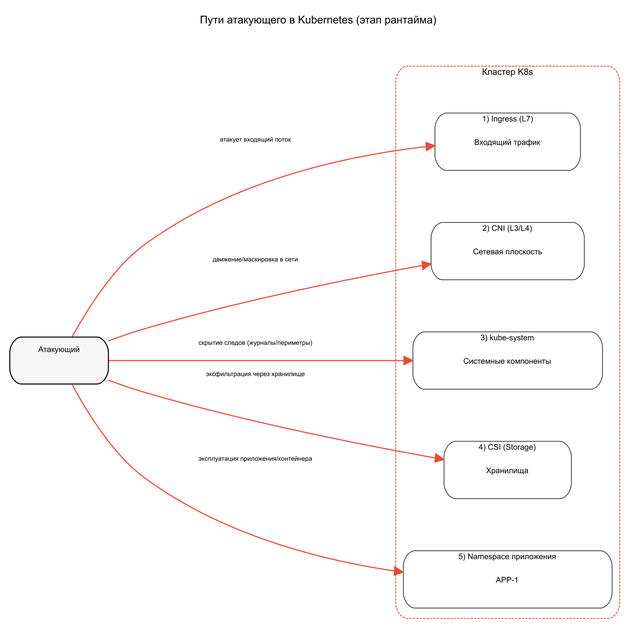
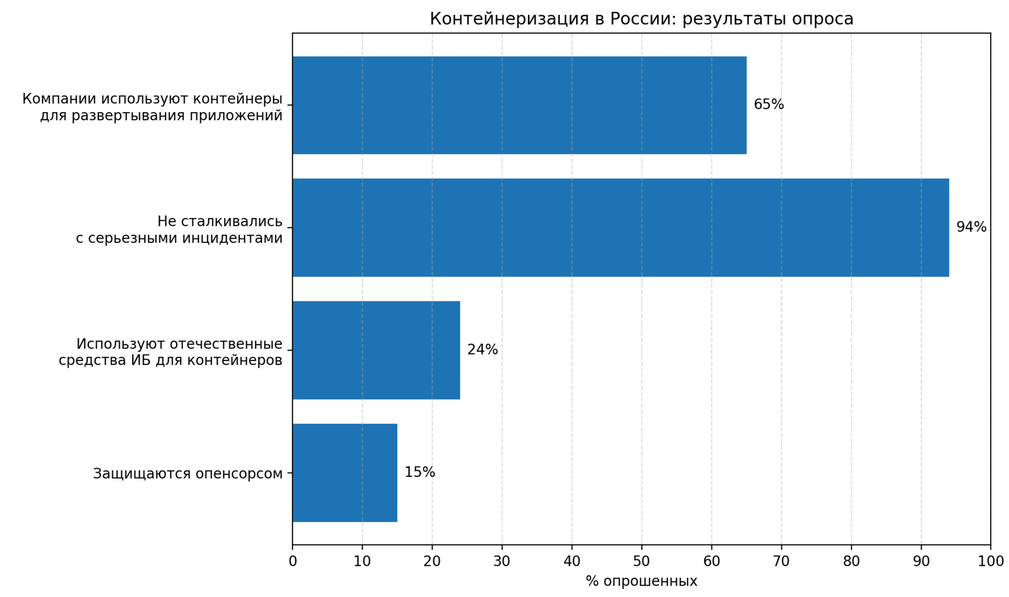
Безопастное развертывание кластеров в инфраструктуре

Атаки на инфраструктуры с Kubernetes

Kubernetes расширяет поверхность атаки: входящий трафик, сетевые плагины (CNI), системные неймспейсы kube-system, хранилища, а также права через RBAC. Ошибки конфигурации дают путь к эскалации и утечкам. Cluster-Audit помогает находить риски до инцидента.

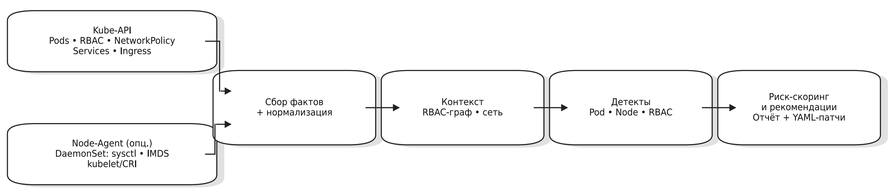




Контейнеризация:

- **65%** компаний используют контейнеры для развертывания приложений
- **94%** не фиксировали серьезных инцидентов, часто из-за нехватки видимости
- **24%** используют отечественные средства ИБ для контейнеров
- **15%** используют open-source

Принцип работы и цель проекта



- Автоматизировать аудит безопасности кластера прямо изнутри (in-cluster).
  - Найти мисконфиги Pod/Node, опасные права RBAC и сетевые дыры.
- Сформировать отчет и рекомендации (вплоть до YAML-патчей) для hardening.

Ключевые проверки

- Pod/Container: privileged, hostPath, hostNetwork/hostPID, runAsNonRoot, allowPrivilegeEscalation, seccomp/AppArmor, drop capabilities.
- RBAC: wildcard (*), доступ к secrets, pods/exec, nodes; цепочки эскалации через create/patch.
- Network: отсутствие deny-by-default, открытые NodePort/Ingress, опасный egress (в т.ч. к IMDS).
- Node (DaemonSet): sysctl hardening, доступность IMDS, параметры kubelet/CRI.
- Выход: HTML + JSON/SARIF, риск-скоринг, доказательства, рекомендации.

Аналоги

- kube-bench (CIS)
- kubeaudit / Polaris
- Trivy Operator / Kubescape
- kube-hunter
- Falco / Tetragon (runtime)

Как использовать

Добавить worker-а в кластер:

```
$ kubectl apply -f audit-job.yaml
```

Запустить и получить отчет:

```
$ kubectl -n audit logs job/k8s-audit > report.json
```

Интерфейс in-cluster инструмента

```
$ kubectl apply -f audit-job.yaml
job.batch/cluster-audit created

$ kubectl logs -f job/cluster-audit
[+] Cluster-Audit v0.1  | mode: job  | namespace: all
[+] Collecting facts: pods, rbac, netpol, services, ingress...
[+] Running detectors (best-effort)...

Summary:  CRITICAL 1  HIGH 3  MEDIUM 4  LOW 6

CRITICAL  K8S-RBAC-007  ClusterRoleBinding/* -> secrets (list,get)
HIGH      K8S-POD-001   privileged container: dev/debug
HIGH      K8S-NET-003   namespace dev: no deny-by-default egress
HIGH      K8S-NODE-002  IMDS reachable from node: worker-1
```

Сбор информации об уязвимостях кластера с логированием и последующей упаковкой в JSON

Плюсы

- **Развертывание внутри кластера**
- **минимум зависимостей**
- **best-effort сбор фактов**
- **понятный отчет с риском.**

Минусы

- **Не заменяет рантайм-детект и пентест**
- **качество зависит от прав и видимости в кластере.**

Выбор технологий

- Язык: **Go**
- API: **client-go**
- Деплой: Docker
- Отчеты: HTML, JSON, SARIF
- Сбор фактов: K8s API + node agent
- Стенд: **kind**